Deep Learning & Neural Network Fundamentals

Week 5 Assignment - Task 1 | Domain: Artificial Intelligence | 6-Week AI Internship

1. What is Deep Learning?

Deep Learning (DL) is a branch of Machine Learning that uses **artificial neural networks with many layers** to learn patterns directly from data. Instead of a programmer hand-crafting features, each layer learns a progressively more abstract representation: in an image model, early layers detect edges, middle layers detect shapes and the last layers detect objects. DL needs large datasets and computing power (GPUs), and powers image recognition, speech assistants, translation and medical diagnosis.

2. AI vs Machine Learning vs Deep Learning

	Artificial Intelligence	Machine Learning	Deep Learning
Meaning	Any technique that makes machines act intelligently	Subset of AI: systems learn from data instead of fixed rules	Subset of ML: multi-layer neural networks
Features	Rules or learned	Often hand-crafted by humans	Learned automatically
Data needed	Varies	Small to medium	Large
Examples	Chess engines, expert systems	Linear regression, decision trees, SVM	CNNs, RNNs, Transformers

AI is the broad field, ML sits inside AI, and DL sits inside ML (AI > ML > DL).

3. What is an Artificial Neural Network (ANN)?

An ANN is a computing model loosely inspired by the brain. It consists of connected units called **neurons** arranged in **layers**. Each connection has a **weight**, and the network learns by adjusting these weights so that its predictions get closer to the correct answers.

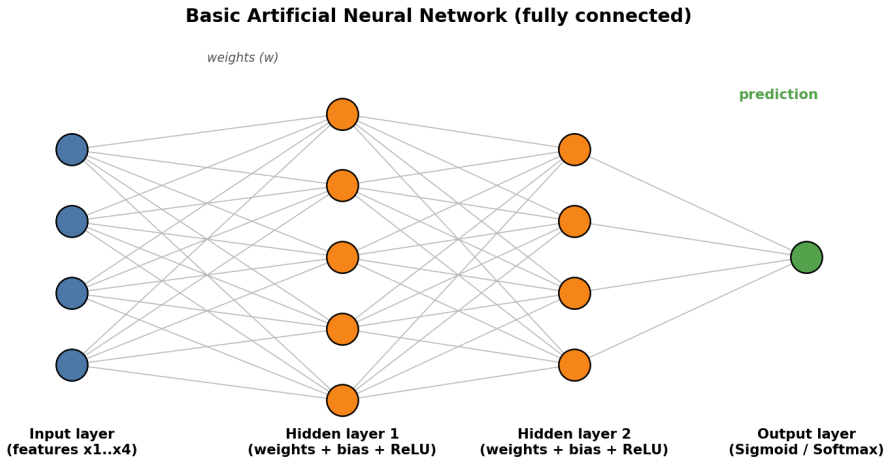


Figure 1: A basic neural network with an input layer, two hidden layers and an output layer.

4. What is a Neuron?

A neuron is the basic unit of a network. It multiplies each input by a weight, adds the results together with a bias, and passes the sum through an activation function: **$z = w_1x_1 + w_2x_2 + \ldots + b$** and **output = f(z)**.

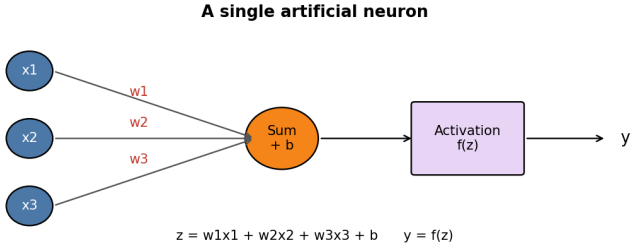


Figure 2: Inside a single neuron.

5. Layers of a Neural Network

Layer	Role	Example (Breast Cancer model)
Input layer	Receives the raw features. One node per feature; no computation.	30 nodes (30 tumour measurements)
Hidden layer(s)	Do the learning. Each neuron computes weighted sum + bias + activation. "Deep" means many hidden layers.	Dense(16, ReLU) and Dense(8, ReLU)
Output layer	Produces the final prediction. Size and activation depend on the task.	1 node, Sigmoid = probability of benign

6. Weights and Bias

Weights control how strongly each input influences a neuron: a large positive weight amplifies an input, a negative weight suppresses it. **Bias** is an extra constant added to the sum that lets the neuron shift its output up or down, much like the intercept in a line equation. Weights and biases are the *learnable parameters*; training means finding good values for them. Our model has 641 of them.

7. Activation Functions

An activation function adds **non-linearity**. Without it, stacked layers collapse into a single linear equation and could not learn complex patterns.

Function	Formula	Output range	Typical use
ReLU	f(z) = max(0, z)	0 to infinity	Hidden layers (fast, default choice)
Sigmoid	$f(z) = 1 / (1 + e^{-z})$	0 to 1	Binary classification output
Softmax	$e^{z_i} / \text{sum}(e^{z_j})$	0 to 1, sums to 1	Multi-class output
Tanh	$(e^{z} - e^{-z}) / (e^{z} + e^{-z})$	-1 to 1	Hidden layers (older / RNNs)

8. Epoch, Loss and Optimizer

Epoch: one complete pass of the entire training dataset through the network. Training normally runs for many epochs (e.g., 50). The data are usually fed in small *batches* within each epoch.

Loss: a single number measuring how wrong the predictions are; lower is better. Examples: *binary cross-entropy* for two classes, *categorical cross-entropy* for many classes, and *mean squared error* for regression.

Optimizer: the algorithm that updates the weights and biases to reduce the loss. It uses gradients computed by *backpropagation* and a *learning rate* (step size). Common optimizers: SGD, RMSprop and **Adam** (used in our projects).

9. Basic Working Flow

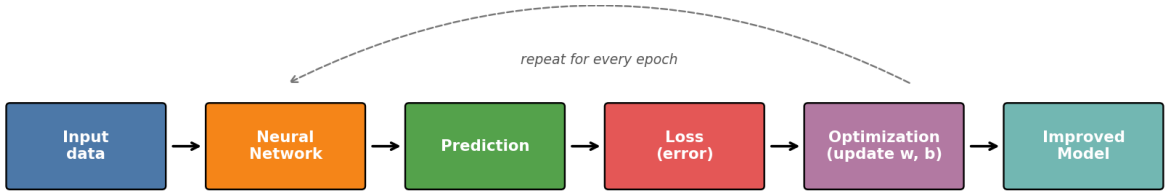


Figure 3: The training loop of a neural network.

Step by step: (1) The input data enter the input layer. (2) **Forward pass:** each layer computes weighted sums and activations until the output layer gives a **prediction**. (3) The **loss function** compares the prediction with the true label and gives an error value. (4) **Backpropagation** calculates how much each weight contributed to the error, and the **optimizer** nudges every weight and bias in the direction that reduces the loss. (5) The result is an **improved model**. This cycle repeats for every batch and every epoch until the loss stops improving, at which point the model is evaluated on unseen test data.